SEGUNDA PARTE — EJERCICIOS (para imprimir y contestar a lápiz)

Nombre: Daniel Fernandez Mejia Equipo: _____ Fecha: 22/sep/2026

Sección A0 — Repaso de Python (encierra la letra)

A0.1. En Python, PUERTO = 5050 crea:

a) una función b) una variable de tipo entero c) una librería d) un socket

A0.2. b"hola" es un valor de tipo:

a) str b) int c) bytes d) bool

A0.3. El ciclo while True: termina cuando:

a) llega a 10 vueltas b) se ejecuta un break c) se acaba la memoria d) nunca puede terminar

A0.4. conexion, direccion = servidor.accept() funciona porque accept() devuelve:

a) una lista b) un diccionario c) una tupla de dos elementos d) un bool

A0.5. Un entorno virtual (venv) sirve para:

a) hacer Python más rápido b) aislar las librerías y versiones de cada proyecto c) conectarse a internet d) cifrar el código

Sección A — Opción múltiple (encierra la letra)

A1. Un sistema distribuido se caracteriza porque sus componentes:

a) comparten la misma memoria RAM b) están en computadoras conectadas por red y se comunican solo por paso de mensajes c) corren en la misma máquina d) no necesitan protocolos

A2. En el modelo cliente-servidor:

a) el servidor inicia la comunicación b) el cliente inicia y el servidor espera en una dirección conocida c) ambos inician al mismo tiempo d) no hay direcciones conocidas

A3. ¿Cuántos bytes puede representar un valor de 8 bits (un byte)?

a) 8 b) 127 c) 256 d) 1024

A4. El número ASCII 10 corresponde a:

a) el espacio b) la letra A c) el salto de línea (\n) d) el dígito 0

A5. El puerto bien conocido de HTTPS es:

a) 80 b) 22 c) 443 d) 53

A6. La llamada accept() devuelve:

a) solo la dirección del cliente b) un socket nuevo para hablar con ese cliente y su dirección c) el número de clientes conectados d) el backlog disponible

A7. Los tres mensajes del handshake TCP, en orden:

a) ACK, SYN, SYN-ACK b) SYN, SYN-ACK, ACK c) SYN, ACK, FIN d) HELLO, SYN, ACK

A8. recv(1024) significa:

a) recibe exactamente 1024 bytes b) recibe exactamente un mensaje c) recibe HASTA 1024 bytes de lo que haya llegado d) recibe 1024 mensajes

A9. En UDP, la dirección del remitente se obtiene con:

a) accept() b) connect() c) recvfrom() d) bind()

A10. Para enviar un ARCHIVO de 3 MB de forma confiable, el transporte adecuado es:

a) UDP, porque es más rápido b) TCP, porque garantiza entrega y orden c) da igual d) ninguno, los archivos no viajan por red

A11. Para video EN TIEMPO REAL, el transporte adecuado es:

a) TCP, porque retransmite los cuadros perdidos b) UDP, porque un cuadro perdido retransmitido llegaría tarde c) da igual d) Telnet

A12. El propósito principal de RPC es:

a) cifrar mensajes b) hacer la red más rápida c) transparencia de acceso: llamar lo remoto como si fuera local d) eliminar el sistema operativo

Sección B — Verdadero o falso (encierra y justifica en la línea)

B1. En un sistema distribuido, una parte puede fallar mientras las demás siguen funcionando; tu programa debe contemplarlo. V / F

Es un fallo parcial, es un rasgo de los sistemas distribuidos

B2. La memoria compartida es el modelo natural para comunicar máquinas distintas por red. V / F

Falso, en red solo es el paso de mensajes

B3. 1 KB equivale a 1000 bytes exactamente. V / F

1024

B4. El carácter ñ ocupa exactamente 1 byte en UTF-8. V / F

No, ocupa 2 bytes

B5. El socket que devuelve accept() es el mismo que hizo listen(). V / F

No, se regresa uno nuevo

B6. TCP garantiza que cada send() corresponde a exactamente un recv(). V / F

No, pq los mensajes pueden llegar pero no están completos

B7. Si un datagrama UDP se pierde, el protocolo lo retransmite automáticamente. V / F

No, no se puede retransmitir

B8. UDP es ideal para aplicaciones que no pueden permitir la pérdida de datos. V / F

No, pq es para aplicaciones que toleran perdidas

Sección C — Relacionar columnas (escribe la letra)

Llamada Tu respuesta Descripción

1. socket(c) _____ a. Asocia el socket a una dirección y puerto locales

2. bind(a) _____ b. Inicia la conexión hacia el servidor (dispara el handshake)

3. listen(e) _____ c. Crea el punto final de comunicación

4. accept(h) _____ d. Envía un datagrama indicando el destinatario cada vez

5. connect(b) _____ e. Pone el socket en modo escucha

6. recvfrom(g) _____ f. Cierra el punto final de comunicación

7. sendto(d) _____ g. Recibe un datagrama y la dirección de quien lo envió

8. close(f) _____ h. Acepta una conexión y devuelve un socket nuevo + dirección

Sección D — Respuesta corta (usa las líneas)

D1. Define con tus palabras “sistema distribuido” y menciona DOS problemas que no existen en una sola máquina.

El sistema distribuido es un conjunto de computadoras que se conectan a través de una red

Los dos problemas serian: fallos parciales y latencia

D2. Explica la regla de oro “ENVIAR = ENCODE · RECIBIR = DECODE” y qué error aparece si se olvida cada mitad.

Los sockets envían bytes, el texto se convierte a bytes al enviarlos y se reconstruye al recibir, si se olvida el encode manda un mensaje de “TypeError”

D3. ¿Por qué el servidor necesita bind() y el cliente no? ¿Qué hace el sistema operativo por el cliente?

Pq el servidor necesita una dirección ip conocida y fija para que lo encuentre, en cambio el cliente no lo necesita ya que con “connect()” el SO le asigna un puerto

D4. Explica el problema del framing y describe DOS soluciones.

TCP entrega un flujo de bytes: un recv() puede traer un mensaje partido o varios cortados. Dos soluciones:

Delimitador: cada mensaje termina con una marca acortada y Ciclo de repetición: llamar recv() hasta completar el mensaje

D5. ¿Qué son stub y skeleton en RPC, en qué máquina va cada uno, y qué es marshalling?

Stub: función local que empaqueta los argumentos, los envia y espera; skeleton: recibe, desempaqueta, ejecuta la función real y devuelve el resultado; marshalling: empaqueta/desempaqueta

Sección E — Lectura de código

E1. ¿Qué imprime exactamente este programa cuando un cliente se conecta y envía "hola"?

```
import socket

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

servidor.bind(("localhost", 5050))

servidor.listen(1)

conexion, direccion = servidor.accept()
```

```
datos = conexion.recv(1024)
print("Me llego:", datos)
print("Tipo:", type(datos))
```

Imprime Me llego: b'hola' y Tipo: <class 'bytes'>

E2. Este código UDP tiene UN error conceptual. Encuéntralo y corrígelo:

```
import socket
servidor = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
servidor.bind(("0.0.0.0", 5050))
servidor.listen(1) # linea con problema
datos, direccion = servidor.recvfrom(1024)
```

El error es en listen(), ya que esa función no existe en UDP

E3. ¿Cuál es el error de este cliente TCP?

```
cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
cliente.connect(("localhost", 5050))
cliente.send("hola servidor")
```

send(), pq tiene texto sin codificar, debería de ser cliente.send

Sección F — Ejercicios prácticos (en tu computadora; entrega capturas)

Cada ejercicio usa los pasos de ejecución en dos terminales del código correspondiente (Tema 20). Verifica la salida esperada antes de contestar.

F1. Ciclos de vida (Códigos 1 y 2). Ejecuta el eco TCP. En una copia del código, subraya:

(a) la línea donde el servidor se bloquea esperando cliente; (b) la línea donde se bloquea esperando datos; (c) la línea del cliente que dispara el handshake.

F2. Ciclo while True + funciones (Código 1). Modifica el servidor de eco: agrega un contador y una función armar_eco(mensaje, numero) que devuelva ECO #N> mensaje. Prueba con tres mensajes.

F3. UDP y best-effort (Código 4). Ejecuta el experimento de pérdida (apagar el servidor a la mitad). Documenta con captura qué observó el cliente y explica en 3 líneas por qué no hubo ningún error de conexión.

F4. Framing (Código 5). Ejecuta el servidor y cliente de framing. Luego modifica el cliente para enviar TRES mensajes en un solo send() y verifica que el servidor los separa. ¿Qué línea del servidor es la que “sabe” dónde termina cada mensaje?

F5. Bloqueo (Código 3). Cambia ESPERA a 7 segundos, ejecuta y anota el tiempo que midió el servidor. Después agrega conexion.settimeout(5) justo después de accept() y vuelve a correr: ¿qué excepción ocurre y en qué lado?

F6. Código limpio. Reescribe este código cumpliendo TODAS las normas de código limpio (constantes, nombres, comentarios que explican el porqué):

```
s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.bind(('',9000))
s.listen(1)
c,a=s.accept()
d=c.recv(1024)
print(d)
```